Regression Assumptions and Diagnostics

Residual Plots, qqPlot, ncvTest

Durjoy Kumar Sarker

Table of contents

1 Why Check Assumptions? 1

2 The Four Assumptions 2

3 Data and Model 2

4 The Four Diagnostic Plots 3
4.1 Plot 1: Residuals vs Fitted (checks linearity) 4
4.2 Plot 2: Normal Q-Q (checks normality) 5
4.3 Plot 3: Scale-Location (checks constant variance) 5
4.4 Plot 4: Residuals vs Leverage (checks outliers and influential points) 5

5 Day 5 Code Tests: qqPlot and ncvTest 6
5.1 qqPlot() 6
5.2 ncvTest() 7
5.3 Reading your results 7

6 Assumption 1: Independence 8

7 What Bad Patterns Look Like (a Quick Demo) 9

8 What if an Assumption Fails? 10

9 Key Takeaways 10

1 Why Check Assumptions?

A regression model gives you numbers (coefficients, p-values, R^2). But those numbers are only **trustworthy if the model's assumptions hold**.

Intuition: the model draws the best line through the data. If the line captured the real pattern, what is **left over** (the residuals) should look like **pure random noise**. If the leftovers still show a pattern, the model missed something, and its p-values and predictions cannot be trusted.

Residual = actual - predicted.

```
library(car)
library(knitr)
```

2 The Four Assumptions

#	Assumption	How we check it
1	**Independence**	Study design (are observations independent?)
2	**Linearity**	Residuals vs Fitted: should be scattered
3	**Normality** (of residuals)	Normal Q-Q plot: points near the line
4	**Constant variance** (homoscedasticity)	Scale-Location plot

3 Data and Model

```
data <- read.csv("../data/raw/agri_yield_data.csv")
head(data)
```

```
  Crop_Yield Fertilizer_kg Rainfall_mm Soil_Nitrogen
1      5.806        113.96      544.37         47.84
2      7.166        112.53      783.75         40.82
3      6.677         77.01      600.30         35.67
4      6.182         75.02      638.97         49.29
5      5.636         84.13      598.16         45.31
6      5.676         85.77      379.35         52.02
```

```r
dim(data)
```

```
[1] 100   4
```

```r
model <- lm(Crop_Yield ~ Fertilizer_kg + Rainfall_mm + Soil_Nitrogen, data = data)
summary(model)
```

```
Call:
lm(formula = Crop_Yield ~ Fertilizer_kg + Rainfall_mm + Soil_Nitrogen,
    data = data)

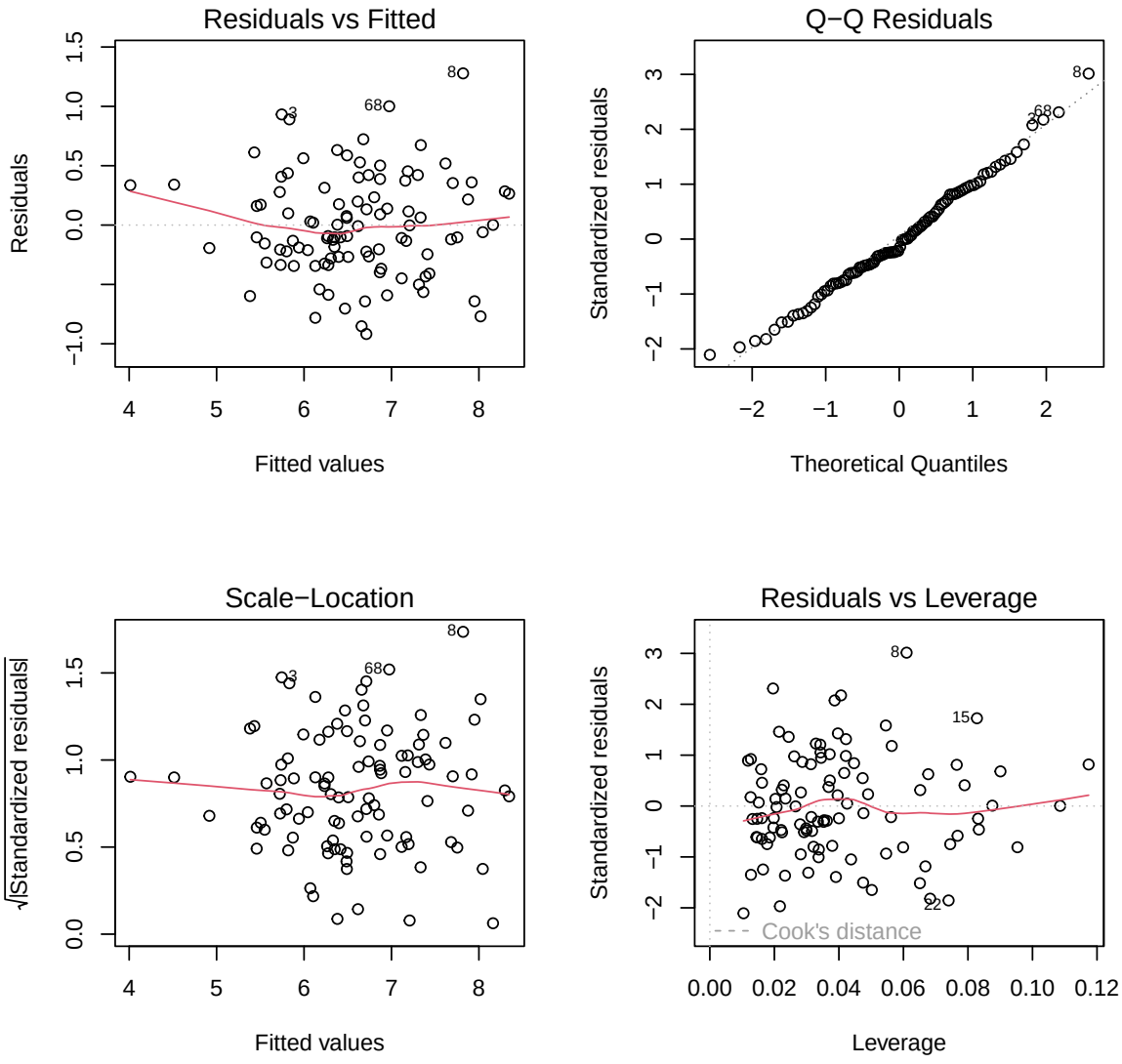
Residuals:
     Min       1Q   Median       3Q      Max
-0.91755 -0.27123 -0.07627  0.31949  1.27801

Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)   0.5500096  0.3548171   1.550    0.124
Fertilizer_kg 0.0210711  0.0021288   9.898 2.46e-16 ***
Rainfall_mm   0.0041579  0.0005395   7.707 1.18e-11 ***
Soil_Nitrogen 0.0301740  0.0051222   5.891 5.67e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4376 on 96 degrees of freedom
Multiple R-squared:   0.78, Adjusted R-squared:  0.7731
F-statistic: 113.5 on 3 and 96 DF,  p-value: < 2.2e-16
```

4 The Four Diagnostic Plots

```r
par(mfrow = c(2, 2))
plot(model)
```




```
par(mfrow = c(1, 1))
```

Now each plot in detail: what is plotted, what the "real meaning" is, what good looks like, and what bad looks like.

4.1 Plot 1: Residuals vs Fitted (checks linearity)

What is plotted: x = the predicted yield, y = the residual (actual - predicted).

The real meaning: if the relationship is truly linear, the model's mistakes should be equally likely to be positive or negative at **every** predicted yield. So the points should form a **random cloud around 0**, and the red line should be roughly flat.

- **What to check:** random scatter, no clear pattern.
- **Bad sign: curved shape** (like a U or an arch) = the true relationship is **non-linear**, and the straight line is systematically over-predicting in some ranges and under-predicting in others.
- **Bad sign: funnel shape** (spread gets wider or narrower) = **non-constant variance** (heteroscedasticity).

4.2 Plot 2: Normal Q-Q (checks normality)

What is plotted: x = where each residual **would** sit if residuals were perfectly normal, y = where it **actually** sits.

The real meaning: it compares your residuals with a perfect bell curve, point by point. If they match, the points fall on the diagonal line.

- **What to check:** points should follow the straight line.
- **Bad sign:** deviations at the **ends** = residuals are not normal (**heavy tails** if both ends bend away, **skewness** if the curve bends one way).
- Small wobbles at the very ends are normal, especially with small samples.

4.3 Plot 3: Scale-Location (checks constant variance)

What is plotted: x = predicted yield, y = square root of the absolute standardized residual (the **size** of the error, ignoring its sign).

The real meaning: it asks, *"does the size of the errors change as predicted yield changes?"* If the errors are equally big everywhere, the red line is flat.

- **What to check:** horizontal spread, flat red line.
- **Bad sign:** increasing or decreasing spread = **variance is not constant**. Example: the model is accurate for low-yield fields but very uncertain for high-yield fields.

4.4 Plot 4: Residuals vs Leverage (checks outliers and influential points)

What is plotted: x = leverage (how **unusual** a row's predictor values are), y = standardized residual (how badly the model fits that row).

The real meaning: a point can hurt the model in two ways. It can be far from the line (big residual, an **outlier**), or it can sit at an extreme predictor value and **pull the line toward**

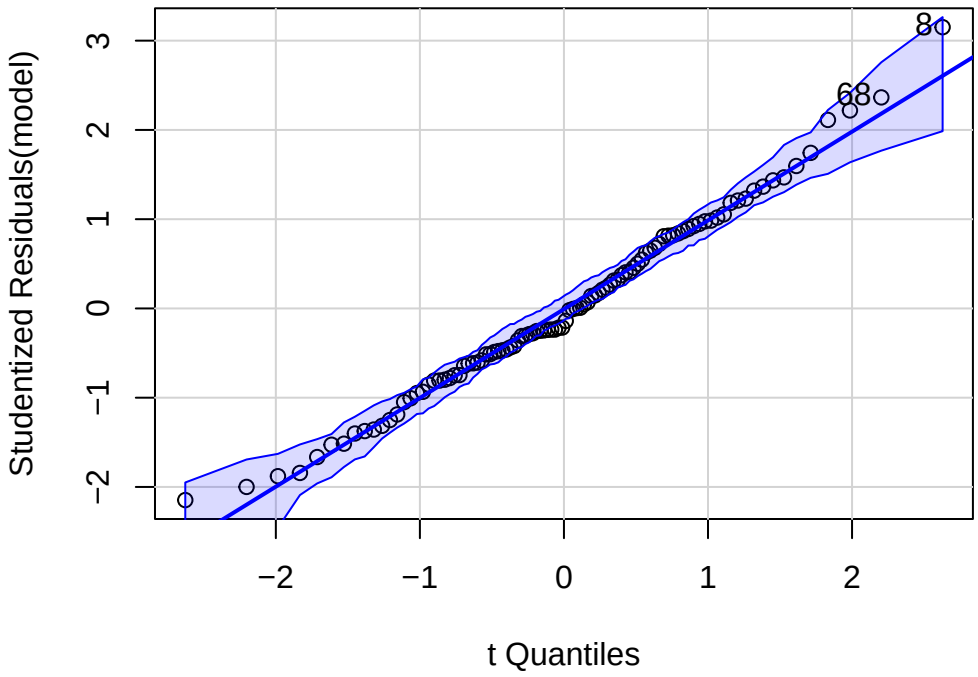
itself (high leverage). A point with **both** is highly **influential**: remove it and the whole model changes.

- **What to check:** outliers or influential points.
- **Bad sign:** points **outside Cook's distance** lines (the dashed curves in the corners) = highly influential.

5 Day 5 Code Tests: qqPlot and ncvTest

5.1 qqPlot()

```
qqPlot(model)
```




```
[1]  8 68
```

- Points near the line = residuals are normal.
- Far from the line = skewness or heavy tails.
- The **shaded band** is a confidence envelope: points inside the band are consistent with normality. The labeled numbers are the row numbers of the most extreme points.

5.2 ncvTest()

```
ncv <- ncvTest(model)
ncv
```

```
Non-constant Variance Score Test
Variance formula: ~ fitted.values
Chisquare = 1.055866, Df = 1, p = 0.30416
```

- **Null hypothesis:** residual variance is constant.
- **p-value $>$ 0.05:** homoscedasticity (constant variance) is OK.
- **p-value $<$ 0.05:** heteroscedasticity (variance is not constant).

5.3 Reading your results

```
p_ncv <- ncv$p
cat(sprintf("- **Constant variance (ncvTest):** p = %.3f. %s\n",
            p_ncv,
            ifelse(p_ncv > 0.05,
                   "This is above 0.05, so we keep the null: variance looks constant (homosce
                   "This is below 0.05, so variance is NOT constant (heteroscedasticity).")))
```

- **Constant variance (ncvTest):** p = 0.304. This is above 0.05, so we keep the null: variance looks constant (homoscedasticity).

```
sw <- shapiro.test(residuals(model))
cat(sprintf("- **Normality (Shapiro-Wilk on residuals):** p = %.3f. %s\n",
            sw$p.value,
            ifelse(sw$p.value > 0.05,
                   "Above 0.05, so there is no evidence against normal residuals.",
                   "Below 0.05, so residuals depart from normality.")))
```

- **Normality (Shapiro-Wilk on residuals):** p = 0.659. Above 0.05, so there is no evidence against normal residuals.

```
cd <- cooks.distance(model)
n  <- nrow(data)
cat(sprintf("- **Influential points (Cook's distance):** the largest value is %.3f (rows abo
            max(cd), 4 / n, sum(cd > 4 / n), sum(cd > 0.5),
            ifelse(sum(cd > 0.5) == 0,
                   "No point is highly influential.",
                   "Check the flagged rows.")))
```

- **Influential points (Cook's distance):** the largest value is 0.147 (rows above 4/n = 0.040: 7; rows above 0.5: 0). No point is highly influential.

Both tests are **supporting evidence**, not a replacement for the plots. With small samples tests are weak; with huge samples they flag tiny, harmless departures. Always read the plot and the test **together**.

6 Assumption 1: Independence

- Each observation should be **independent** of the others: knowing one field's residual should tell you nothing about another's.
- This is checked by **study design**, not by a plot. Problems appear when observations are linked, for example:
  - plots next to each other (spatial correlation)
  - the same field measured repeatedly over time (time correlation)
  - plots in the same block or farm
- If observations are linked, ordinary `lm()` p-values are too optimistic, and a model that accounts for blocks or time (such as a mixed model) is needed.

For data stored in time or field order, one quick supporting check is the Durbin-Watson test (a value near 2 means no correlation between neighbouring residuals):

```
durbinWatsonTest(model)
```

```
 lag Autocorrelation D-W Statistic p-value
   1      0.05775323      1.833819    0.43
 Alternative hypothesis: rho != 0
```

Only trust this if the **row order** of the data means something (time or position). For a shuffled cross-sectional file it is not informative.

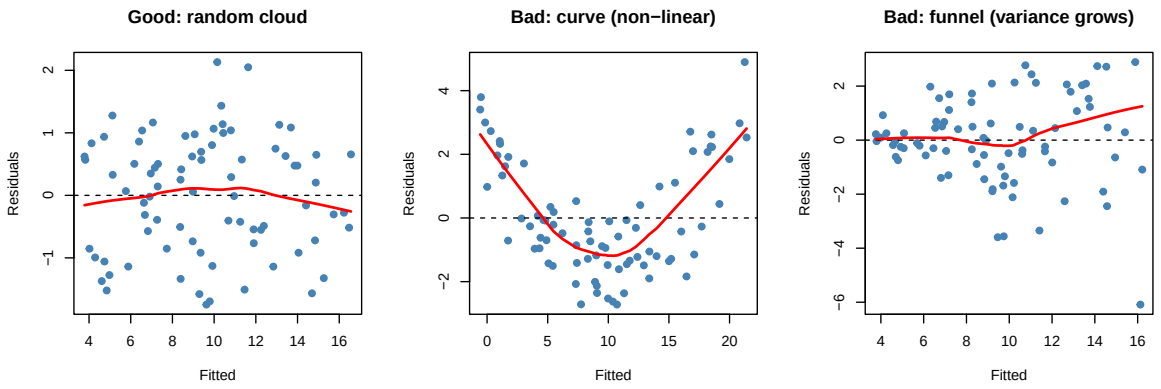
7 What Bad Patterns Look Like (a Quick Demo)

To really see what the warning signs mean, here are three simulated cases. The left one is good, the others are bad.

```
set.seed(10)
x <- runif(80, 1, 10)

y_good   <- 2 + 1.5 * x + rnorm(80, sd = 1)
y_curve  <- 2 + 0.25 * x^2 + rnorm(80, sd = 1)
y_funnel <- 2 + 1.5 * x + rnorm(80, sd = 0.3 * x)

par(mfrow = c(1, 3))
for (case in list(list(y_good, "Good: random cloud"),
                  list(y_curve, "Bad: curve (non-linear)"),
                  list(y_funnel, "Bad: funnel (variance grows)"))) {
  m <- lm(case[[1]] ~ x)
  plot(fitted(m), residuals(m), main = case[[2]],
       xlab = "Fitted", ylab = "Residuals", pch = 19, col = "steelblue")
  abline(h = 0, lty = 2)
  lines(lowess(fitted(m), residuals(m)), col = "red", lwd = 2)
}
```




```
par(mfrow = c(1, 1))
```

- **Good:** points scatter evenly around 0, red line is flat.
- **Curve:** the red line bends. A straight line is the wrong shape for this data.
- **Funnel:** the spread widens to the right. Errors get bigger for higher predictions.

8 What if an Assumption Fails?

Problem	Typical remedy
Non-linearity (curve)	Add a squared term, or transform a predictor (e.g., log)
Non-normal residuals	Transform the response (log, square root), or check for outliers
Non-constant variance (funnel)	Transform the response (log), or use weighted or robust methods
Influential points	Check for data entry errors first; compare the model with and without the point
Dependent observations	Use a model that accounts for blocks or time (mixed model)

9 Key Takeaways

1. Assumptions: **independence, linearity, normality, constant variance**.
2. The four plots from `plot(model)`:
   - **Residuals vs Fitted:** should be a random cloud (curve = non-linear, funnel = unequal variance).
   - **Normal Q-Q:** points on the line (bends at the ends = non-normal).
   - **Scale-Location:** flat red line (slope = variance changes).
   - **Residuals vs Leverage:** no points beyond Cook's distance.
3. **`qqPlot(model):`** points near the line (and inside the band) = normal residuals.
4. **`ncvTest(model):`** $p > 0.05$ = homoscedasticity; $p < 0.05$ = heteroscedasticity.
5. Residuals are the model's leftovers: **if they look like noise, the model is doing its job**.